10/27: Lecture 18. PSPACE VS. TQBF

1 PSPACE VS. NPSPACE

1.1 Review

Recall that

$$\text{PSPACE} = \{\text{Problems solvable by a DTM using poly-space}\}$$
$$\text{NPSPACE} = \{\text{Problems solvable by an NTM using poly-space}\}$$

Also, by Savitch's Theorem,

$$\text{PSPACE} = \text{NPSPACE}.$$

Currently, we only know that

$$\text{P} \subseteq \text{NP} \subseteq \text{PSPACE} \subseteq \text{EXP}$$
$$\text{coNP} \subseteq \text{PSPACE}$$

1.2 Open Questions

We have strong belief that

$$\text{NP} \cup \text{coNP} \subsetneq \text{PSPACE}.$$

We also do not know whether

$$\text{P} = \text{NP} = \text{PSPACE}.$$

1.3 PSPACE-Completeness

This is analogous to NP-completeness and NL-completeness.

Definition. To show that a language L is *PSPACE-complete*, we need to:

1. Show that $L \in \text{PSPACE}$.
2. Show that $\forall L' \in \text{PSPACE}$, $L' \leq_{\text{P}} L$.

Implications. If L is PSPACE-complete, and we find that $L \in \{\text{P, NP, some subset of PSPACE}\}$, then PSPACE "collapses" to a smaller class.

1.4 Characterizing PSPACE Using Connectivity

Recall from the previous lecture that:

- Directed st-Connectivity asks whether there is a path from s to t in a directed graph.
- Ordinary Directed $st-$Connectivity is NL-complete.

For PSPACE, we instead consider *succinct* directed connectivity:

- Instead of explicitly listing out each vertex and edge, we describe a potentially enormous graph using a compact representation (e.g. a circuit) that determines whether an edge exists.
- Since a poly-space TM can have exponentially many configurations, we can again reduce PSPACE computations as a reachability problem in configuration graphs.

2 Quantified Boolean Formulas

Definition. (Quantified = the use of $\exists$ and $\forall$).

$$\text{QBF} = \text{Quantified Boolean Formula}$$

Examples. Recall that each $x, y \in \{0, 1\}$.

$\exists x, y.[x \vee y]$. (True, let $x = 1$.)

$\forall x, y.[x \vee y]$. (True, let $x = 0, y = 0$.)

$\exists x.[x \wedge \neg x]$. (False.)

$\forall x.[x \vee \neg x]$. (True.)

2.1 True Quantified Boolean Formulas

Definition.

$$\text{TQBF} = \text{True Quantified Boolean Formulas}$$

More formally,

$$\text{TQBF} = \{Q[\phi] : Q[\phi] \text{ is a fully quantified boolean formula and is true.}\}$$

Examples.

$$Q[\phi] = \forall x \exists y \exists z.[(x \Leftrightarrow y) \wedge (y \vee z)]$$

This is True (in TQBF), because for either $x \in \{0, 1\}$, if we choose $y = x$ and $z = 1$, both parts of the formula become True.

$$Q'[\phi] = \forall x \exists y \forall z.[(x \Leftrightarrow y) \wedge (y \vee z)]$$

This is False, because if $x = 0$, then $y = 0$. However, in this case, the statement does not work for $z = 0$ (as $y \vee z$ evaluates to false).

Theorem. TQBF is PSPACE-Complete.

- To prove this, we will show that (1) TQBF $\in$ PSPACE, and (2) $\forall L \in$ PSPACE, $L \leq_{\text{P}}$ TQBF.
- We first show that TQBF $\in$ PSPACE. To solve this determinsitically, we can construct a binary tree and use DFS.

  Consider the general boolean formula

  $$\exists x_1 \forall x_2 \exists x_3 \ldots \forall x_{n-1} \exists x_n.[\phi(x_1, \ldots x_n)].$$

  We can evaluate this formula by branching on every possible value of each variable:

  - If the variable is used with $\exists$, then at least one branch must be True.
  - If the variable is used with $\forall$, then both branches must be True.

  This is poly-space because since there are n variables, the recursion depth is at most n. Moreover, we only need to remember the assignments along the current branch, and we do not need to store the entire binary tree. Thus, TQBF $\in$ PSPACE.

- Next, we want to show that

  $$\forall L \in \text{PSPACE}. L \leq_{\text{P}} \text{TQBF}.$$

  1. First, suppose some $L \in$ PSPACE. Then, there is a DTM that decides L using poly-space.

     We want to construct poly-time reduction R st. $R(x) = Q[\phi]$ and $x \in L \Leftrightarrow Q[\phi]$ is true (i.e. we want to convert the question of whether a TM accepts x into the question of whether a quantified Boolean formula is True).

2. To create this reduction, we can try to use a tableau construction, similar to the Cook-Levin proof for SAT.

   Suppose the machine uses $S(n)$ (poly)-space. However, it may still run for exponentially many steps, i.e. $T(n) \leq 2^{O(S(n))}$. This means that the tableau has $S(n)$ width and $T(n)$ length, meaning writing down the entire tableau would require exponential time.

   Thus, is there some way we can certify the correctness of the boolean formula in a more space-efficient way, if we allow quantifiers (i.e. instead of explicitly writing down every configuration of the computation, can we used quantified Boolean formulas to describe and verify the computation more compactly)?